

Listas e Tuplas

Listas e Tuplas

São **estruturas de dados** fundamentais no Python, que armazenam **coleções de dados**.

Listas e Tuplas

Permitem armazenar qualquer tipo de dado
(*int*, *float*, *str* e até mesmo outras
listas/tuplas).

Listas e Tuplas

Podem ser acessadas por **indexação** (*offset*, ou **deslocamento**).

```
l1 = [1, 2, 3, 4, 5]  
l1[0] # 1  
l1[1] # 2  
l1[2] # 3
```

Sintaxe - Listas

```
# listas  
l1 = [1, 2, 3, 4, 5]  
l2 = ["abacaxi", "banana", "maçã"]  
l3 = [l1, l2]  
l_mix = [1, 2.4, "uva", l3]
```

Sintaxe - Tuplas

```
# tuplas
t1 = (1, 2, 3, 4, 5)
t2 = ("abacaxi", "banana", "maçã")
t3 = (t1, t2)
t_mix = [1, 2.4, "uva", t3]
```

Tuplas vs Listas

A diferença entre as duas estruturas de dados está na mutabilidade: **Listas podem ser modificadas e tuplas não.**

Editar listas funciona 

```
# tuplas  
l1 = [1, 2, 3, 4, 5]  
l1[0] = 200
```

Editar tuplas resulta em erro 

```
# tuplas  
l1 = [1, 2, 3, 4, 5]  
l1[0] = 200
```


Tentar editar tuplas resultará no erro a seguir:

```
Traceback (most recent call last):  
  File "/Users/arxgsf/Projects/Ufpr/CI185/02 - Tratamento de Dados/tup.py", line 2, in <module>  
    t1[0] = 10  
    ~~~^  
TypeError: 'tuple' object does not support item assignment
```

Em outras palavras:

“ Listas são objetos mutáveis e tuplas não. ”

Quando utilizar cada uma ?

Utilidade de Tuplas

1. Garantir a **imutabilidade/integridade** dos dados.
2. Retorno múltiplo de funções.
3. Chaves de dicionários (*dict*).
4. Desempenho de memória (tuplas são **levemente** mais eficientes que listas)

Imutabilidade

```
# Registro: (ID, Nome, Preço, Em_Estoque)
produto = (101, "Teclado Mecânico", 250.00, True)
id_prod, nome, preco, em_estoque = produto
```

Retorno Múltiplo

```
def obter_ponto():  
    x1 = float(input())  
    y1 = float(input())  
    return x1, y1 # Retorna a tupla (1920, 1080)  
  
p1 = obter_ponto()  
x1, y1 = obter_ponto() # unpacking de tupla
```

Chave de Dicionários

```
# Mapeamento de coordenadas para nomes de locais
locais = {
    (-25.4284, -49.2733): "Curitiba",
    (-23.5505, -46.6333): "São Paulo"
}
```

Tentar usar uma lista causaria:

```
TypeError: unhashable type: 'list'
```

Utilidade de Listas

1. Manter um registro de coleções de dados de tamanhos variados.
2. Maior flexibilidade com **insert**, **append**, **remove**, **pop** etc.
3. Ordenação e manipulação de dados.

Manipulação de Listas

Listas podem crescer e encolher de tamanho durante a execução do programa.

```
frutas = ["Banana"]  
  
frutas.append("Uva")  
frutas.append("Pêra")  
  
frutas.remove("Uva")  
  
#frutas irá conter: ["Banana", "Pêra"]
```

Operações Com Listas

Operação	Descrição
<code>l = []</code>	lista vazia
<code>l = [1, 2]</code>	lista c/ valores
<code>l = list('pão')</code>	lista de iterable
<code>l = list((1, 2, 3))</code>	lista de iterable
<code>l = list(range(4))</code>	lista de iterable

Operação	Descrição
<code>l[i]</code>	indexação
<code>len(l)</code>	tamanho
<code>l1 + l2</code>	concatenação
<code>l * 3</code>	repetição
<code>for item in l</code>	iteração

Operação	Descrição
<code>l[i] = x</code>	atribuição indexada
<code>x in l</code>	pertencimento
<code>l.append(x)</code>	ins. ao final
<code>l.insert(i, x)</code>	ins. no índice i

Operação	Descrição
<code>l.pop(i)</code>	remove do final
<code>l.remove(x)</code>	remove x
<code>del l[i]</code>	remove no índice i

Operação	Descrição
<code>l.index(x)</code>	busca x
<code>l.count(x)</code>	conta ocorrências de x
<code>l.sort(x)</code>	ordenação
<code>l.reverse</code>	inversão
<code>l.copy()</code>	cria cópia
<code>l.clean()</code>	limpa

Exercício:

Calcular o n° de acertos de um aluno em uma prova de 10 questões com cinco alternativas cada (A, B, C, D e E).

Entrada:

Gabarito e cartão resposta do aluno no
formato "ADBSDCSBDC"

ADBSDCDBDC

ABBSCCCBDC

Saída:

Número de acertos do aluno.

Exercício:

Descubra se um dado é viciado: lance-o n vezes e **determine o número de ocorrências de cada face.**

Exercício:

Receber uma string `s` e invertê-la.

1. Proibido utilizar `s.reverse`.
2. Proibido iterar por `s` utilizando `for`.
3. Proibido utilizar indexação `s[i]`.